Data Analytics using Python - Project (Flask)

❖ **What is Flask?**

- Flask is a lightweight Python web framework.
- Flask helps you create a website/web app using Python.
- With Flask you can:
    - Create web pages
    - Take user input (forms, buttons, uploads)
    - Run your ML model in backend
    - Show predictions on a webpage

❖ **How Flask connects with Machine Learning?**

- Normally, in ML you do this:

  model.predict(data)
- But that runs only in Python file.
- Flask can:
    - Take input from a web page
    - Send it to ML model
    - Get prediction
    - Show result on browser
- So, your ML project becomes a **real application (not just code).**

❖ **Flow in ML + Flask project :**

User → Web Page → Flask → ML Model → Result → Web Page

❖ **Example:**

- User enters:
    - Pclass, Sex, Age, Fare
- Flask sends it to:
    - Trained ML model
- Model returns:

- Survived / Not Survived
- Flask shows it nicely.

❖ **Why Flask is used in ML projects?**

✓ Very easy to learn

✓ Pure Python

✓ Good for students & research projects

✓ Used in real industry too

✓ Perfect for dashboards & prediction apps

❖ **Other Options :**

1. Streamlit (BEST for beginners & ML dashboards)
2. FastAPI (modern & very fast)
3. Django (full web application)
4. Gradio (for quick ML demos)

❖ **Typical ML + Flask project structure :**

```
Project_folder/
│
├── app.py        (Flask main file & ML Model)
├── templates/index.html  (HTML files)
├── static/      (CSS, images)
```

❖ **Flask = bridge between your ML model and real users.**

❖ **ML Project Creation using Flask and Python :**

Step 1: Create Project Folder

- A **project folder** is just a normal folder you create in your computer where you keep all files of your Flask app together.
- You can create it **anywhere**

1: Create folder on Desktop

1. Go to **Desktop**
2. Right click → **New** → **Folder**
3. Name it: **titanic_flask**

   Now this is your **project folder**.

2: Open that folder in VS Code

1. Open **VS Code**
2. Click **File** → **Open Folder**
3. Select **titanic_flask** folder

   Now VS Code will show your folder in left side.

3: Create required files inside folder

Inside folder create:

- ✓ app.py
- ✓ titanic.csv (Data Set)
- ✓ templates folder
  - Inside templates → index.html

Your folder should look like:

```
titanic_flask/
│
├── app.py
├── titanic.csv
└── templates/
    └── index.html
```

➢ **Why project folder needed?**

- Because Flask app needs:
  - app.py

- templates/ folder
- dataset file

All should be in one place so Flask can find them correctly.

Step 2: Install Libraries

- Open terminal inside folder:

  pip install flask pandas scikit-learn

- For this:

  Step 1: Open your project folder in VS Code

  Example: **titanic_flask**

  Step 2: Open Terminal

  In VS Code menu bar:

  Terminal → New Terminal

  A terminal will open at the bottom.

  Step 3: Install libraries

  Type and press Enter:

  pip install flask pandas scikit-learn

➔ **For Check if pip is working :**

Run: **pip --version**

If it shows version, pip is installed

➔ **Common error: 'pip' not recognized**

If you get error like:

'pip' is not recognized as an internal or external command

Then try:

python -m pip install flask pandas scikit-learn

Step 3: app.py (All-in-one code)

- Create file **app.py** in project folder.

```
from flask import Flask, render_template, request
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

app = Flask(__name__)

# Load dataset and train model
df = pd.read_csv("titanic.csv")

# Keep only required columns
df = df[["Survived", "Pclass", "Sex", "Age", "Fare"]]

# Fill missing Age
df["Age"] = df["Age"].fillna(df["Age"].median())

# Convert Sex to numeric
df["Sex"] = df["Sex"].map({"male": 1, "female": 0})   [Like Dictionary Replacement]

# Features and target
X = df[["Pclass", "Sex", "Age", "Fare"]]
y = df["Survived"]

# Train model
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
[It forces Python to split the data in the same way every time.]
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

```
# Flask routes
@app.route("/", methods=["GET", "POST"])
def home():
    prediction = ""
    if request.method == "POST":
        pclass = int(request.form["pclass"])
        sex = int(request.form["sex"])   # 1 for male, 0 for female
        age = float(request.form["age"])
        fare = float(request.form["fare"])
        result = model.predict([[pclass, sex, age, fare]])[0]
        prediction = "Survived" if result == 1 else "Not Survived"
    return render_template("index.html", prediction=prediction)
if __name__ == "__main__":
    app.run(debug=True)
```

This code trains model automatically and runs flask.

➔ **Code Explanation :**

- **Load all Libraries -**
  - Flask → creates web app
  - render_template → shows HTML page
  - request → gets form data from user
  - pandas → load & handle dataset
  - train_test_split → divide data into training/testing
  - LogisticRegression → ML algorithm for classification

- **Create Flask App –**

  app = Flask(__name__) ← This starts your web application.

- **Load Titanic dataset –**

  df = pd.read_csv("titanic.csv") ← Reads CSV file into data frame.

- **Flask route (web page) –**

  ```
  @app.route("/", methods=["GET", "POST"])
  def home():
  ```

  This means:

  When user opens website "/" page, run this function.

- **Take user input from form** –

  ```
  if request.method == "POST":
      pclass = int(request.form["pclass"])
      sex = int(request.form["sex"])
      age = float(request.form["age"])
      fare = float(request.form["fare"])
  ```

- **Make prediction –**

  ```
  result = model.predict([[pclass, sex, age, fare]])[0]
  ```

  Model predicts:

  0 → Not survived

  1 → Survived

- **Convert result to words –**

  ```
  prediction = "Survived" if result == 1 else "Not Survived"
  ```

- **Send result to webpage –**

  ```
  return render_template("index.html", prediction=prediction)
  ```

  It displays output as:

  HTML page + prediction text

- **Run Flask app –**

```
if __name__ == "__main__":
    app.run(debug=True)
```

Starts Flask Web Server and you can open in browser: http://127.0.0.1:5000/

Step 4: Create HTML Page

Create file: **templates/index.html** in project folder.

```
<!DOCTYPE html>
<html>
<head>
  <title>Titanic Survival App</title>
</head>
<body>
  <h1>Titanic Survival Prediction</h1>
  <form method="POST">
    <label>Pclass:</label>
    <select name="pclass">
      <option value="1">1st Class</option>
      <option value="2">2nd Class</option>
      <option value="3" selected>3rd Class</option>
    </select>
    <br><br>
    <label>Sex:</label>
    <select name="sex">
      <option value="1">Male</option>
      <option value="0">Female</option>
    </select>
    <br><br>
    <label>Age:</label>
    <input type="number" step="0.1" name="age" required>
```

```
            [Allow only number with decimal point in gap of 0.1]
        <br><br>
        <label>Fare:</label>
        <input type="number" step="0.1" name="fare" required>
                <br><br>
        <button type="submit">Predict</button>
    </form>
    <h2>{{ prediction }}</h2>
</body>
</html>
```

Step 5: Run Flask App

In terminal:

python app.py

Open browser:

http://127.0.0.1:5000/

❖ **Add /dashboard page**

Step 1: Install extra libraries (for charts)

Open terminal in your project folder and run:

pip install matplotlib seaborn

Step 2: Update Project Structure

Your folder should now look like:

```
titanic_flask/
├── app.py
├── titanic.csv
├── templates/
│   ├── index.html
│   └── dashboard.html
```

```
└── static/
    ├── survival_count.png (After run the code)
    └── gender_survival.png
```

You must create a folder named static inside project.

Step 3: Update app.py (Add dashboard route)

Replace your **app.py** with this:

```
from flask import Flask, render_template, request
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

import matplotlib
matplotlib.use("Agg")   # (Anti-Grain Geometry) important for Flask server
#[create image files instead of pop-up windows]
import matplotlib.pyplot as plt
import seaborn as sns
import os

app = Flask(__name__)

# Load dataset and train model
df = pd.read_csv("titanic.csv")

df = df[["Survived", "Pclass", "Sex", "Age", "Fare"]]
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Sex"] = df["Sex"].map({"male": 1, "female": 0})

X = df[["Pclass", "Sex", "Age", "Fare"]]
y = df["Survived"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Home route (Prediction)
@app.route("/", methods=["GET", "POST"])
def home():
 prediction = ""
 if request.method == "POST":
   pclass = int(request.form["pclass"])
   sex = int(request.form["sex"])  # 1 male, 0 female
   age = float(request.form["age"])
   fare = float(request.form["fare"])
   result = model.predict([[pclass, sex, age, fare]])[0]
   prediction = "Survived" if result == 1 else "Not Survived"
return render_template("index.html", prediction=prediction)

# Dashboard route
@app.route("/dashboard")
def dashboard():
    # create static folder if not exists
    if not os.path.exists("static"):
        os.makedirs("static")

    # Chart 1: Survival count
    plt.figure()
    sns.countplot(x="Survived", data=df)
    plt.title("Survival Count (0=Died, 1=Survived)")
    plt.savefig("static/survival_count.png")
    plt.close()
```

```
    # Chart 2: Gender vs Survival
    plt.figure()
    sns.countplot(x="Sex", hue="Survived", data=df)
    plt.title("Survival by Gender")
    plt.savefig("static/gender_survival.png")
    plt.close()

  # Stats
  total = len(df)
  survived = int(df["Survived"].sum())
  died = total - survived
  survival_rate = round((survived / total) * 100, 2)

  return render_template( "dashboard.html", total=total, survived=survived,
died=died, survival_rate=survival_rate)

if __name__ == "__main__":
  app.run(debug=True)
```

Step 4: Create dashboard.html

Create file: **templates/dashboard.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>Titanic Dashboard</title>
</head>
<body>
  <h1>Titanic Dashboard</h1>
  <h3>Summary</h3>
  <p><b>Total Passengers:</b> {{ total }}</p>
  <p><b>Survived:</b> {{ survived }}</p>
```

```
    <p><b>Died:</b> {{ died }}</p>
    <p><b>Survival Rate:</b> {{ survival_rate }}%</p>
    <hr>
    <h3>Charts</h3>
    <img src="/static/survival_count.png" width="400">
    <img src="/static/gender_survival.png" width="400">
    <br><br>
    <a href="/">← Back to Prediction</a>
</body>
</html>
```

Step 5: Update index.html (Add dashboard link)

Update your **templates/index.html** by adding this line:

```
<br>
<a href="/dashboard">Go to Dashboard</a>
```

Full final **index.html:**

```
<!DOCTYPE html>
<html>
<head>
    <title>Titanic Survival App</title>
</head>
<body>
    <h1>Titanic Survival Prediction</h1>
    <form method="POST">
        <label>Pclass:</label>
        <select name="pclass">
            <option value="1">1st Class</option>
            <option value="2">2nd Class</option>
            <option value="3" selected>3rd Class</option>
        </select>
```

```
        <br><br>
        <label>Sex:</label>
        <select name="sex">
            <option value="1">Male</option>
            <option value="0">Female</option>
        </select>
        <br><br>
        <label>Age:</label>
        <input type="number" step="0.1" name="age" required>
        <br><br>
        <label>Fare:</label>
        <input type="number" step="0.1" name="fare" required>
        <br><br>
        <button type="submit">Predict</button>
    </form>
    <h2>{{ prediction }}</h2>
    <br>
    <a href="/dashboard">Go to Dashboard</a>
</body>
</html>
```

Step 6: Run Application

Run:

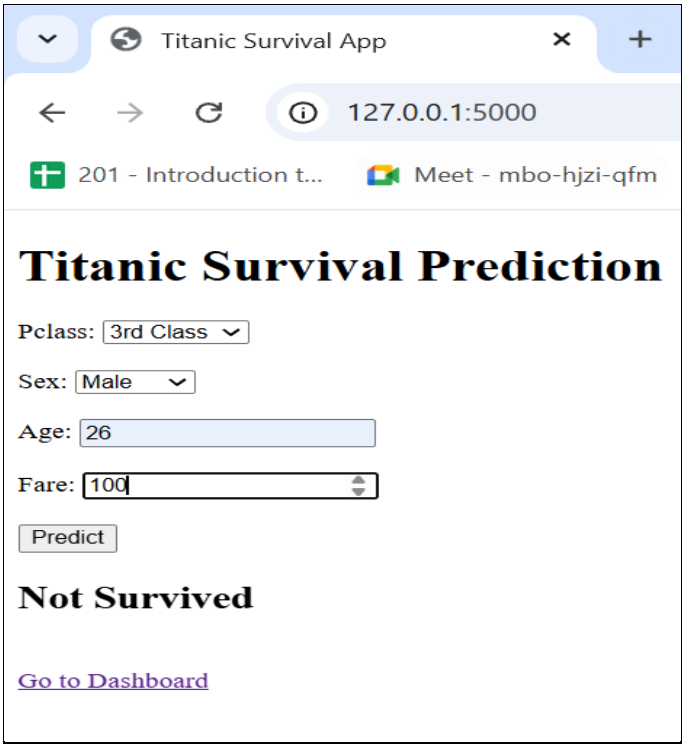
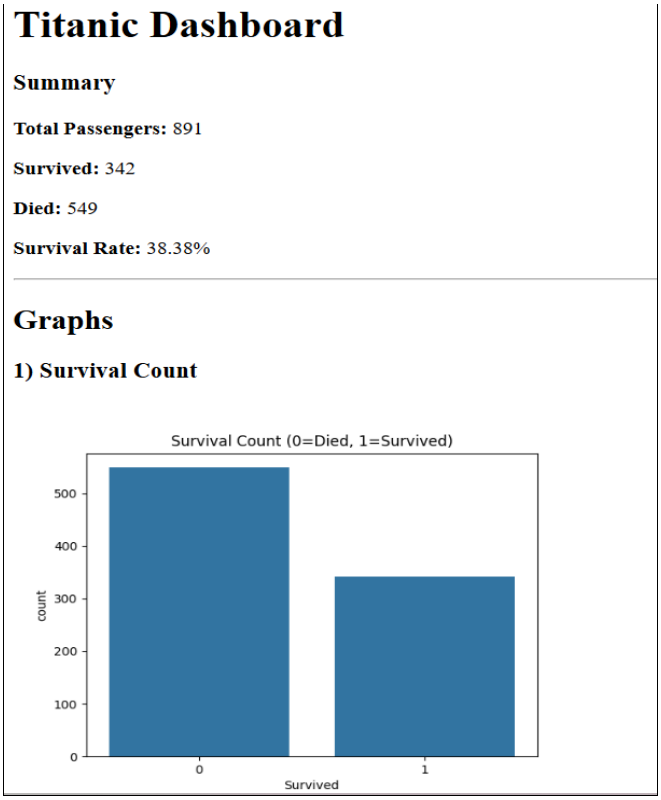
python app.py

Open:

Prediction page: http://127.0.0.1:5000/

Dashboard page: http://127.0.0.1:5000/dashboard

Prediction Page



Dashboard Page